Cluster Computing (2025) 28:403
https://doi.org/10.1007/s10586-024-05091-1

Sfedrl-ids: secure federated deep reinforcement learning-based intrusion detection system for agricultural internet of things

Rabaie Benameur[1] · Amine Dahane[1,2]

Received: 7 August 2024 / Revised: 3 November 2024 / Accepted: 26 December 2024 / Published online: 16 June 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

This paper presents an innovative framework designed to enhance the security of agricultural Internet of Things (IoT) systems. The Secure Federated Deep Reinforcement Learning Intrusion Detection System (SFEDRL-IDS) is a highly efficient solution that leverages Federated Double Deep Q-learning (FDDQN) to collectively train a network intrusion detection system (NIDS) in a distributed manner. The confidentiality of local model weights is safeguarded through the application of Fully Homomorphic Encryption (FHE) and Cryptographic Keys over Polynomial Rings (CKKS). To effectively mitigate IoT attacks, the FELIDS system employs three distinct Deep Learning (DL) classifiers: Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), and Long Short-Term Memory networks (LSTM), which collaboratively counteract threats targeting agricultural IoT environments. The results not only demonstrate the superiority of the SFEDRL-IDS system compared to traditional centralized machine learning techniques but also highlight its effectiveness in preserving the privacy of agricultural IoT systems. The SFEDRL-IDS system achieves remarkable accuracy rates of 98.67% for multiclass classifiers and 97.73% for binary classifiers. Furthermore, its minimal CPU and memory resource utilization attests to its efficiency. Additionally, we deploy the proposed framework within an intelligent irrigation system, with results further validating its effectiveness.

Keywords IoT · IDS · Deep reinforcement learning · DDQN · Federated learning · Deep learning

Abbreviations

IoT	Internet of things
IDS	Intrusion detection system
NIDS	Network intrusion detection system
NTA	Network traffic analysis
AI	Artificial intelligence
ML	Machine learning
DL	Deep learning
FL	Federated learning
MDP	Markov decision process
RL	Reinforcement learning
DRL	Deep reinforcement learning
DQN	Deep Q-learning network
DDQN	Double deep Q-learning network
CNN	Convolutional neural network‘
LSTM	Long short-term memory
DNN	Deep neural network

✉ Rabaie Benameur
benameur.rabaie@edu.univ-oran1.dz

Amine Dahane
dahane.amine@univ-oran1.dz

1 University of Oran 1, Institute of Applied Science and Technology ISTA, Oran, Algeria

2 Image, Signal and Intelligent Systems (LiSSi) Laboratory, TincNET Research Team, Univ. Paris-Est Créteil, Créteil, France

1 Introduction

Growing security concerns have made cybersecurity indispensable across industries. Consequently, organizations must implement robust mechanisms to protect their databases and network infrastructures [1]. The proliferation of Internet of Things (IoT) networks has increased diverse and covert network attacks, posing a significant challenge to network security [2]. In response to these challenges, researchers have developed and deployed Network Intrusion Detection Systems (NIDS), which monitor and identify potentially malicious activities in the emerging realm of computer networks referred to as Software-Defined Networking (SDN) [3]. NIDS plays a crucial role in

reinforcing network security. Network Intrusion Detection Systems (NIDS) can be categorized into two primary types: signature-based detection and anomaly-based detection. Signature-based detection involves comparing network traffic patterns with a predefined database of known malicious patterns to identify potential attacks. Anomaly-based detection, on the other hand, uses machine learning (ML) models to classify network traffic as normal or malicious based on a training dataset with relevant features and labels. This study focuses exclusively on anomaly-based detection using the reinforcement learning (RL) approach. RL has gained traction for detecting and categorizing network attacks through autonomous agents. These RL agents learn network behaviors within predefined environments and develop future defensive strategies by receiving feedback from their surroundings through trial and error, enhancing their capabilities by rewarding favorable actions and penalizing undesirable ones. This approach allows RL to generate training data through interactions between the agent and its environment, reducing the dependence on extensive labeled datasets typically required by supervised ML algorithms. Recent advancements in network intrusion detection techniques have leveraged RL [4, 5]. However, these approaches face challenges distinguishing legitimate network traffic from malicious activity and managing large centralized datasets. The state explosion problem arises when RL agents encounter many learning states. To address these limitations, Deep Reinforcement Learning (DRL) techniques such as Deep Q-learning (DQN) and Double Deep Q-learning (DDQN) have been introduced [6]. DDQN employs neural networks (NN) to tackle the state explosion problem effectively. Additionally, the Federated Learning (FL) framework enables decentralized training of deep neural networks while maintaining data confidentiality [7]. However, current literature on FL often involves openly sharing local model weights with aggregator nodes, assuming these nodes are trustworthy, which is not valid in real IoT systems [8]. Despite numerous DRL-based IDS techniques proposed in the literature, practical implementations and deployments in real IoT environments still need to be improved [9]. This limitation is primarily due to the heavy reliance on publicly available datasets for training and assessment, often neglecting the critical aspects of data acquisition and the availability of actual network traffic source files. As a result, a significant gap exists between theoretical research and practical implementation of NIDS in real-world environments [10]. The contributions of this work can be summarized as follows:

- Improved NIDS performance by integrating Double Deep Q-learning (DDQN) with Federated Learning (FL), enabling decentralized local agent training.
- Secure the aggregation and exchange of encrypted model weights with the server, utilizing Fully Homomorphic Encryption (FHE) employing Cryptographic Keys over Polynomial Rings (CKKS).
- Conducted a comparative analysis between centralized and Federated Deep Reinforcement Learning (FDRL) approaches with different deep learning (DL) models (CNN, LSTM, DNN) and evaluated their performance using various metrics.
- We practically implement an IDS using open-source software and thoroughly evaluate its efficiency in a real-world context to validate its functionality and performance. Crucially, this assessment confirms that the IDS implementation does not negatively impact the performance of the target IoT systems.

This paper is structured as follows: Sect. 2 delivers a thorough and insightful review of the latest advancements in NIDS. Section 3 offers an in-depth analysis of the SFEDRL-IDS framework, along with a detailed discussion of its implementation. Section 4 presents rigorous experimentation and performance evaluations that compellingly demonstrate the effectiveness of our approach. Finally, Sect. 5 draws significant conclusions and outlines promising future research directions.

2 Related works

In this section, we explore recent categories of NIDS from the literature, discussing relevant works within each category. We analyze the significant features and contributions identified in these studies. Finally, we contextualize our approach based on these distinct features and contributions.

2.1 NIDS solutions for IoT network security

Zhang et al. [11] propose ICVAE-BSM, a novel intrusion detection method for IoT that integrates deep learning with statistical approaches. In a conditional variational autoencoder (CVAE), this method changes the encoder's output probability distribution to learn the posterior distributions of different sample classes. Additionally, it incorporates the Borderline Synthetic Minority Oversampling Technique (BSM) to address dataset imbalance issues. We further refine the model by fine-tuning it using both original and generated samples, which significantly enhances the accuracy of IoT attack detection, especially for unbalanced datasets. Chohra et al. [12] describe an optimization method that combines swarm intelligence with ensemble methods to deal with problems that can't be predicted. Their method achieves notable performance improvements over existing machine learning models, as evidenced by

high F1 scores: 92% on NSL-KDD, 92.9% on UNSW-NB15, and 97.3% on IoT-Zeek datasets. Lee et al. [13] present CoNNIDS, an IDS framework designed to effectively detect ambiguous attacks and learn new attack types. This system has two parts: the unknown attack detection (UAD) module uses an autoencoder, and the malicious behavior parsing (MBP) module uses a deep neural network (DNN). We introduce an agile training mechanism to enhance efficiency by selectively retraining low-impact neurons. According to the evaluation results, CoNNIDS is better at finding unknown attacks and needs less retraining than traditional IDS methods. Javeed et al. [14] developed a deep learning-based intrusion detection system (IDS) tailored for edge-centric situational awareness (SA) in extreme environments. The hybrid IDS combines bidirectional gated recurrent units and long short-term memory with a softmax classifier to detect attacks at the network edge. It employs truncated backpropagation through time (TBPTT) to speed up learning for long network sequences. The authors also present an attack scenario and deployment architecture for SA. Experimental results on the CIC-IDS2018, ToN-IoT, and Edge-IIoTset datasets highlight the IDS's superiority over traditional and advanced methods.

2.2 Federated learning-based intrusion detection system

Friha et al. introduce FELIDS [15], a Federated Learning Intrusion Detection System for securing agricultural IoT infrastructures. FELIDS uses a local learning approach where devices share model updates exclusively with an aggregation server to ensure data privacy and maintain raw data confidentiality. Experimental results show FELIDS outperforms traditional centralized methods in protecting IoT device data privacy and achieving high detection accuracy. Javeed et al. [16] introduce a horizontal federated learning (HFL) model that combines CNN and BiLSTM for intrusion detection. This hybrid approach improves IoT intrusion detection by using CNNs for spatial feature extraction and BiLSTM for capturing temporal dependencies. The system follows a zero-trust model, keeping data on local edge devices and sharing only learned weights with a centralized FL server. Experimental results on CICIDS2017 and Edge-IIoTset show the method's superiority over centralized and federated deep learning-based IDS. Rabaie et al. [17] propose a novel personalized federated learning framework designed to detect cyber attacks in IoT networks. In their approach, they leverage Knowledge Distillation (KD) to personalize the local models, where the global model acts as the teacher model and the local models serve as student models. This method allows for effective knowledge transfer, enhancing the performance of individual local models while maintaining privacy. The findings demonstrate the effectiveness of the proposed framework across both binary and multi-attack scenarios, highlighting its robustness and adaptability in addressing diverse cyber threats within IoT environments. Yazdinejad et al. [18] proposed another recent work that introduces Block Hunter, a federated learning-based framework designed to detect attacks in blockchain-enabled IIoT networks. Block Hunter employs a cluster-based architecture with various machine learning models, achieving high accuracy in anomaly detection while efficiently minimizing bandwidth consumption.

2.3 Deep reinforcement learning-based intrusion detection system

Lopez-Martin et al. [19] introduce deep reinforcement learning (DRL) algorithms for intrusion detection using labeled datasets, utilizing a pseudo-environment to sample training intrusions and evaluating models such as DQN, DDQN, PG, and AC. DDQN shows improved detection performance and faster inference times than traditional methods. Suwannalai et al. [5] explore DRL's effectiveness in NIDS with the AE-DQN algorithm, achieving 80% accuracy and a macro F1 score of 79%. Tharewal et al. [4] optimize IIoT intrusion detection using DL and RL, achieving a 99% detection rate with the DRL-IDS system employing PPO2 and LightGBM. Our work aligns with recent innovations in the field, as summarized in Table 1, focusing on creating a comprehensive dataset from open-source Pcap files for IoT systems, covering diverse attacks. We implement Federated Deep Double Q-learning with CNNs, LSTMs, and DNNs, ensuring privacy through CKKS fully homomorphic encryption for direct aggregation on encrypted local weights. The IDS implementation utilizes modern open-source tools with meticulous performance evaluation and testing.

3 Secure federated deep reinforcement learning-based intrusion detection system (SFEDRL-IDS)

This section provides a detailed exploration of the SFEDRL-IDS framework, a key tool in understanding and mitigating potential security threats and vulnerabilities within agricultural IoT networks.

3.1 Security threats in modern agriculture

Modern agriculture 4.0, with its integration of digital technologies and automation, brings with it a host of risks and vulnerabilities. This includes IoT devices, sensors,

Table 1 Related studies on intrusion detection systems in IoT environments

References	Datasets	Attacks	Algorithms	Learning strategy	Implementation
[16]	CICIDS2017, and Edge-IIoTset	DDOS attacks, HTTP Flooding, Uploading attack, Backdoor, Ransomware, MITM, Port scanning, and Password attack	CNN-BiLSTM	Federated	Simulation
[11]	NSL-KDD, CIC-IDS2017, and CSE-CIC-IDS2018	DoS, Probe, R2L, U2R, Benign, Bot, Infiltration, Portscan, Web Attack, and Bruteforce	ICVAE-BSM	Centralized	Simulation
[13]	NSL-KDD, and CIC-IDS2017	DoS, Probe, R2L, U2R, Benign, Bot, Infiltration, Portscan, Web Attack, and Bruteforce	CoNNIDS	Centralized	Simulation
[15]	CSE-CIC-IDS2018, MQTTset, and InSDN	DoS, Probe, R2L, U2R, Benign, Bot, Infiltration, Portscan, Web Attack, Bruteforce, and others	CNN, RNN, and DNN	Federated	Simulation
[17]	Edge-IIoTset	DDOS attacks, HTTP Flooding, Uploading attack, Backdoor, Ransomware, MITM, Port scanning, and Password attack	Knowledge Distillation	Federated	Simulation
SFEDRL-IDS	Edge-IIoTset	DDOS attacks, HTTP Flooding, Uploading attack, Backdoor, Ransomware, MITM, Port scanning, and Password attack	DDQN, CNN, LSTM, DNN, and FHE approach known as CKKS	Federated	Experimentation

actuators, and data analytics. The increasing adoption of IoT and data-driven agriculture practices underscores the urgent need for robust privacy and security measures across the data lifecycle. Traditional network threats exploit protocol vulnerabilities, including malware injections for system control and DDoS attacks that disrupt operations. Ensuring comprehensive security measures and protocol adherence is crucial for protecting agricultural data integrity, confidentiality, and availability. Physical access to agricultural sensors and devices poses security risks, allowing for potential compromise and operational disruptions. The harsh environmental conditions in agriculture further challenge digital technology integration, impacting equipment reliability and communication stability, leading to measurement inaccuracies and data loss.

3.2 Adversary model and defense objectives

In IoT systems, intruders can originate from internal or external sources and carry out network attacks via the internet. These attacks include DDoS attacks utilizing botnets, unauthorized access to IoT services through password brute-force attacks on FTP or SSH protocols, and the exploitation of system vulnerabilities, such as remote code execution, to inject malware or ransomware. Unauthorized access can lead to database data manipulation attacks. Conversely, internal intruders within the network have the capability to execute more advanced attacks, including Man-in-the-Middle (MITM) attacks. The primary objective of the proposed SFEDRL-IDS framework is to identify emerging cyber attacks originating from both external and internal sources that target the IoT infrastructure. Early detection of these attacks allows for timely and appropriate countermeasures to mitigate their impact. Our assumptions take into account several factors. First, IoT devices have been previously using deep learning techniques to detect anomalies in environmental factors data, as established in our prior work [20]. In contrast to related research, which requires a secure cloud environment free from malicious aggregator nodes, the aggregation process is protected through the use of HFE techniques.

3.3 System design goals

The global architecture of SFEDRL-IDS is illustrated in Fig. 1. The agent is an autonomous entity interacting with an environment to learn and make decisions based on rewards and punishments. We employ neural networks as a function approximation method to tackle the complexity of network traffic analysis, where the number of possible states is extensive. This allows us to overcome the need to store each state's values or policies. Initially, an offline RL training approach is employed, wherein agents utilize a local dataset to optimize their performance efficiently. In this context, new Pcap files that store network packet

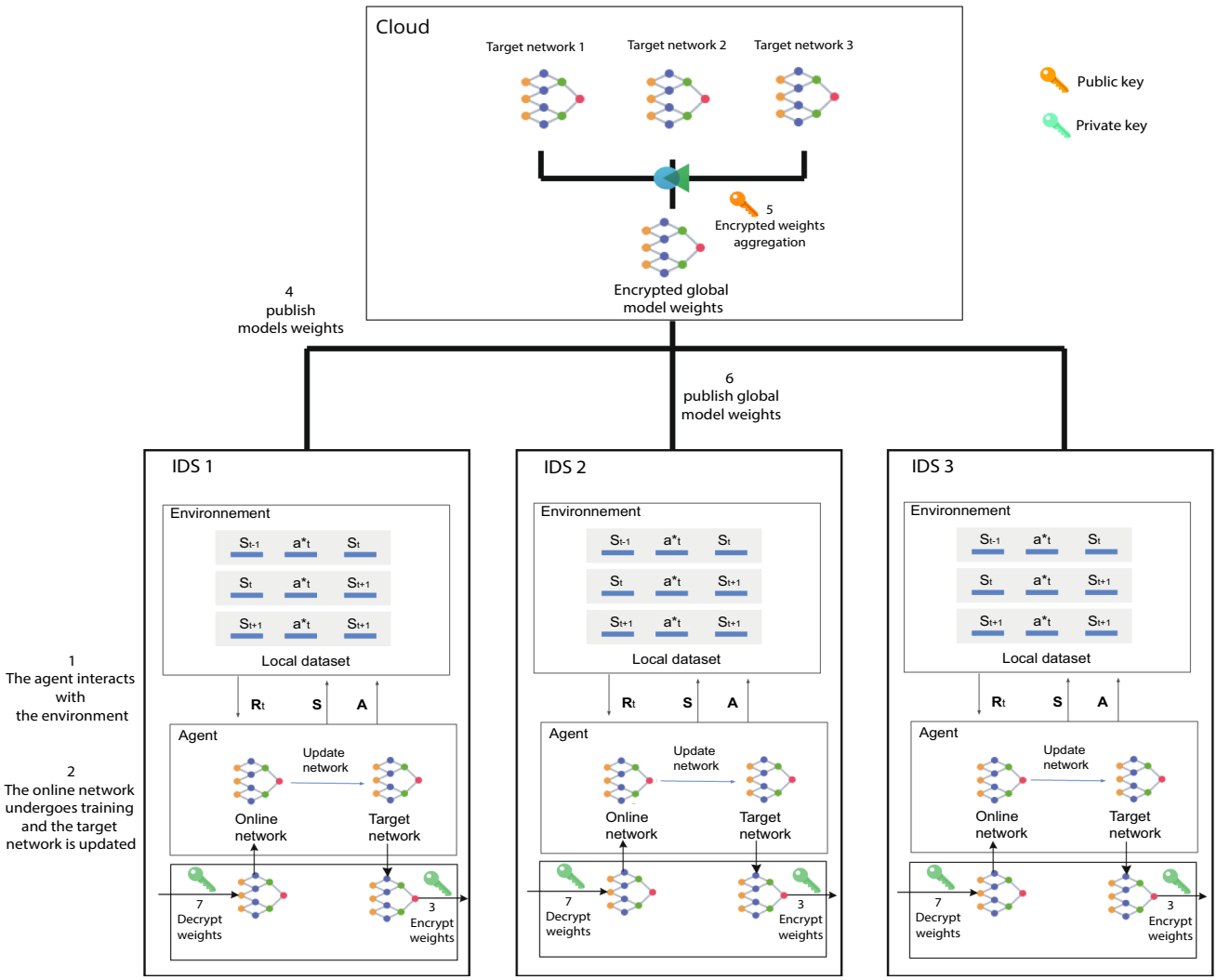


Fig. 1 Global architecture of DDQNFL-IDS

capture data related to an IoT agricultural system are utilized to build a time series dataset that exhibits the same distribution as the target system. The application of the FL approach involves agents collaborating to distribute and train a robust global model capable of accurately detecting various types of attacks. Periodically, the secured weights of the target network are published to a cloud platform for aggregation, forming a global model that is subsequently disseminated to all participating entities. This iterative process continues indefinitely, enabling the system to adapt to its dynamic nature.

As shown in Fig. 2, network traffic flows into IoT gateways equipped with a NIDS that monitors and analyzes the incoming data. These gateways are essential for capturing and preprocessing network traffic, extracting vital features such as packet size, source and destination IP addresses, protocols, and other pertinent attributes. The extracted features are organized into a structured format that serves as the input state for the classifier. The preprocessed network stream is then input into the neural network, which is the foundation of the DDQN agent. This neural network analyzes the input state and classifies the network traffic into specific categories, which can include normal traffic or various types of intrusions. After a classification decision is made, it goes through a verification process by the Security Operations Center (SOC). The SOC is a specialized team tasked with real-time monitoring, analysis, and incident response, responsible for reviewing and validating the identified traffic category. This verification enhances the system's reliability and aids in detecting and addressing any false positives or negatives. Once verified, the labeled traffic is incorporated into the training dataset, creating a tuple that consists of the state, action, and label. The DDQN agent then receives a reward (R) based on how accurate its classification was. This reward acts as feedback, allowing the agent to assess the

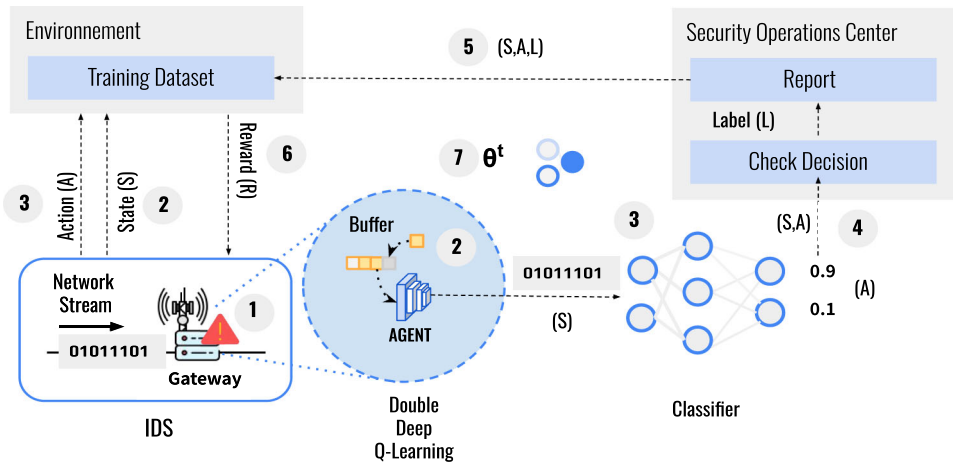


Fig. 2 Architectural overview of local NIDS training using DDQN

effectiveness of its chosen action. The agent subsequently utilizes this reward signal to update the online neural network, continuously improving the classifier's ability to recognize new patterns and evolving threats. This iterative training and feedback process ensures that the NIDS remains adaptable and responsive to changes in network environments, enabling it to learn from new data and enhance its detection capabilities over time.

3.4 Data collection

In the current state of the art, most research papers depend on publicly available datasets, which present difficulties when deploying IDS in real environments. Additionally, the constraints imposed by the dataset itself limit our ability to tailor the data collection process. Our research paper leverages an open-access pcaps file that contains data collected from IoT agricultural environments [8]. The dataset encompasses information generated by over 10 IoT devices, including low-cost digital sensors for temperature, humidity, water level detection, PH, and soil moisture. The pcaps files specifically focus on the most prevalent attacks targeting IoT systems. We classify these attacks into five main threat categories: DoS/DDoS attacks, information gathering, MITM attacks, injection attacks, and malware attacks. The proposed approach involves periodic aggregation and statistical extraction of network flows and protocols. This strategy aims to decrease raw data processing and inference times during deployments. Reducing the frequency of model inferences in production mode optimizes computational resources, resulting in improved resource utilization.

Algorithm 1 Data collection process

```
Input: Pcap_file (the path to the Pcap file)
Open the Pcap file using a traffic analyzer.
Initialize variables to hold statistics related to packets, protocols, and traffic types.
while Packets are remaining do
    Read a packet.
    if the packet is None then
        Exit the loop.
    end if
    Update the total number of packets, total size, and other relevant variables.
    if a time interval has passed then
        Calculate various statistics related to packet lengths and traffic types.
        Store the calculated statistics in a dictionary (row).
        Append the dictionary row to the list rows.
        Reset the variables for the next time interval.
    end if
end while
return List of rows containing the calculated statistics.
```

In our study, we employed the sliding windows technique [21], a method utilized to analyze multivariate time series data. All extracted features are shown in Table 2. This technique involves dividing the time series into overlapping windows of a predetermined fixed length, sequentially sliding through the data.

$$\text{Window}(i) = \{X[i], X[i+1], \ldots, X[i+k-1]\} \quad (1)$$

This approach effectively captures the temporal dynamics of the data while simultaneously ensuring computational efficiency.

Table 2 Description and details of each local IoT IDS dataset

Categories	Attacks	Samples	Training samples	Testing samples
IDS 1	Normal	2213	1991	221
	DDoS TCP SYN Flood Attacks	395	356	39
	DDoS ICMP Flood Attacks	310	279	31
	DDoS UDP Flood Attacks	301	271	30
	DDoS HTTP Flood Attacks	200	180	20
IDS 2	Normal	2213	1991	221
	Backdoor attack	4,345	3,911	434
	Ransomware attack	3,309	2,978	331
	MITM (ARP spoofing + DNS) Attack	228	205	23
IDS 3	Normal	2213	1991	221
	Port Scanning attack	6,875	6,188	687
	Password attacks	9,981	8,983	998
	Backdoor attack	4,345	3,911	434

3.5 Data preparation

The collected dataset was divided into various subsets based on cybersecurity threats. Furthermore, the dataset was separated into training and testing sets to assess the effectiveness of the IDS model, as depicted in Table 2.

3.6 Reinforcement learning (RL)

In decision-making scenarios involving agent-environment interactions, modeling the system as a Markov Decision Process (MDP) with discrete time steps is crucial. The MDP is denoted by a 5-tuple, $M = (S, A, T, R, \gamma)$, where S represents possible states, A denotes available actions, T is the transition function defining state transitions, R assigns values to state-action pairs, and γ is a discount factor weighing future rewards. A policy π determines action selection probabilities [22]. Solving the MDP finds the optimal policy π^* maximizing cumulative rewards [23]. Q-learning updates action values $Q(s, a)$ iteratively to converge to $Q^*(s, a)$ [24]. Deep Q-Networks (DQN) use neural networks to handle high-dimensional states [23]. In scenario, when monitoring network traffic based on multiple features, such as connection counts and protocol types. The resulting state space can be vast, calculated as m^k (where m is the number of values each feature can take, and k is the number of features). This expansion results in excessive memory usage for the Q-table, longer training times, and challenges in generalizing across states. In contrast, DDQN also employs neural networks to approximate Q-values, allowing it to efficiently manage larger state spaces. Additionally, it addresses overestimation bias by using separate networks for action selection and value estimation, which improves accuracy in attack detection.

Algorithm 2 Double deep Q-learning network based on federated learning approach

Input: Replay memory D_c, online network Q_c, target network $\hat{Q}_c$
Output: Trained DDQN
for $round = 1$ to $Rounds$ **do**
  for $c = 1$ to C **do**
    for $episode = 1$ to M **do**
      for $step = 1$ to $steps$ **do**
        Select network traffic category a using ϵ-greedy policy based on Q_c and current state s
        Choose network traffic category a and calculate the reward r and next state s'
        Store tuple (s, a, r, s') in D_c
        Sample random minibatch of tuples (s_i, a_i, r_i, s'_i) from D_c
        Set target value $y_i = r_i + \gamma \max_{a'} \hat{Q}_c(s'_i, a'; \theta)$
        Update online network weights.
        Every M steps, update target network weights $\hat{Q}_c \leftarrow Q_c$
        Update current state $s \leftarrow s'$
      end for
    end for
  end for
  Encrypt the target networks $\hat{Q}_c$ weights using a shared CKKS private key.
  Target networks encrypted $\hat{Q}_c{}^{\text{Cipher}}$ weights aggregation:

$$\theta_{\text{avg}} = \theta - \eta \times \frac{\sum(\nabla(E(\theta_c; D_c)))}{N}$$

  Receive and decrypt the aggregated encrypted weights of the global model using the shared private key, and then synchronize the weights of the local model

$$\hat{Q}_c \leftarrow (1 - \tau) \cdot \hat{Q}_c + \tau \cdot \theta_{\text{avg}}$$

.
end for

DDQN's time complexity is $O(N \cdot T)$ for local training-where N is the number of experiences and T is the number of training iterations-while its space complexity remains $O(N)$. This structure supports real-time threat detection and enables rapid responses to evolving threats, unlike traditional Q-learning, which requires complete retraining. Overall, DDQN effectively overcomes the challenges posed by the state explosion problem found in Q-learning, making it a superior choice for adaptive intrusion detection systems.

3.7 Secure federated deep reinforcement learning-based IDS

A standard DDQN setup consists of an online network and a target network with identical architectures. The FL methodology entails training the target network models using a collection of active IDS clients, as illustrated in Algorithm 2. At the start of each round, the active IDS clients engage in interaction and training within their local environments. CKKS's shared private key encrypts local model weights after several steps. A central server subsequently synchronizes the encrypted weights. The central server aggregates encrypted updates from each IDS client using the corresponding public key. All IDS clients receive these aggregated weights and can decrypt them using their shared private key. We repeat this process indefinitely to ensure adaptability in complex environments. The online and target neural networks are very important in the DDQN algorithm because they work as function approximators and handle problems that come up with large and continuous state spaces well. The model architectures used in the proposed solution are presented in Table 3.

Careful selection of suitable neural network architectures allows the system to detect intricate patterns efficiently. A lot of research has shown that Deep Neural Networks (DNN), Long Short-Term Memory networks (LSTM), and Convolutional Neural Networks (CNN) have a lot of potential for dealing with complicated collections of time series data that have many variables. The NIDS continuously monitors incoming network traffic and performs data pre-processing tasks. The DDQN predicts Q-values for all actions based on the current state and ultimately chooses the action with the highest Q-value as its decision. The IDS swiftly triggers an intrusion alert whenever it detects an intrusion. On the other hand, if it detects no intrusion, it continues to monitor subsequent network traffic. This real-time inference and decision-making process empowers the IDS to respond proactively to potential threats and attacks. To address the challenges posed by non-IID data, which closely mirrors real-world scenarios in federated learning, we focus on its effects on model convergence and accuracy in NIDS. To mitigate these issues, we implement several effective strategies. One key technique is Gradient Clipping of Model Weights, which stabilizes training by limiting gradients during updates. In the NIDS context, non-IID data can generate large gradient values due to diverse client traffic patterns. Clipping these gradients prevents extreme values from skewing the model's learning trajectory, mathematically represented as [25]:

$$g' = \begin{cases} g & \text{if } \|g\| \leq t \\ t \cdot \dfrac{g}{\|g\|} & \text{if } \|g\| > t \end{cases} \tag{2}$$

where g is the original gradient, g' is the clipped gradient, and t is the threshold. This method promotes balanced updates, enhancing both convergence speed and accuracy in detecting various attack patterns. Additionally, we employ Personalized Models with DDQN (Federated Personalized Model approach) to tailor Q-values to the unique data distributions of individual clients. This approach allows each model to better capture the characteristics of a client's network traffic, improving generalization and reducing overestimation bias. As a result, this leads to higher detection rates for diverse network attacks, ensuring that SFEDRL-IDS remains robust and adaptive.

Table 3 DDQN online and target networks architecture

Layer	Model outputs
DNN-based model	
Input	45 × 12
Dense	45 × 12 × 128
Dense	45 × 12 × 64
Dense	45 × 12 × 32
Flatten	17280 × 1
Dense	11
LSTM-based model	
Input	45 × 12
LSTM	45 × 128
LSTM	61 × 1
Flatten	64 × 1
Dense	32 × 1
Dense	11
CNN-based model	
Input	45 × 12
Conv2D	43 × 10 × 16
Conv2D	41 × 8 × 32
Global max pooling 2D	32 × 1
Dense	32 × 1
Dense	16 × 1
Dense	11

3.8 Cryptographic keys over polynomial rings (CKKS)

FL enables collaborative machine learning model training among multiple participants while maintaining the privacy of their respective training data. However, transmitting model updates without encryption could empower adversaries to infer sensitive information. Despite the encryption of communication channels, the entity in charge of aggregation still has access to the unencrypted model updates. Additional security measures are imperative to prevent the unauthorized deduction of private data. We employed a method called Fully Homomorphic Encryption (FHE) that makes Cryptographic Keys over Polynomial Rings (CKKS) possible. This method allows to apply many addition and multiplication operations encrypted weights [26]. The CKKS scheme is specifically designed to facilitate arithmetic operations on encrypted data while preserving a level of precision that is suitable for the secured manupulation of neural networks weights [13]. The process begins by encoding plaintext data into a polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ through an encoding function $\mathcal{E}$. For a given plaintext vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]$, CKKS encodes this vector as follows [27]:

$$\mathbf{x} \xrightarrow{\mathcal{E}} \mathcal{E}(\mathbf{x}) = \sum_{i=0}^{N-1} x_i X^i (\text{mod q}) \tag{3}$$

In our implementation, we set the polynomial modulus degree N to 8192. This choice significantly affects both the capacity and size of the resulting ciphertexts, thereby influencing computational efficiency and security levels. To ensure precision during the encoding process, CKKS employs a global scale denoted as Δ. In our configuration, we establish this scale at $\Delta = 2^{21}$. The encoding function can be expressed mathematically as:

$$\mathcal{E}(\mathbf{x}) = \lfloor \Delta \cdot \mathbf{x} \rfloor \tag{4}$$

The global scale is crucial for balancing computational precision and managing noise growth during homomorphic operations. Selecting an appropriate scale is vital for controlling the inherent noise introduced by the encryption process. After encoding the plaintext, it undergoes encryption using a public key pk, resulting in the ciphertext $\mathbf{c}$:

$$\mathbf{c} = \text{Enc}(pk, \mathcal{E}(\mathbf{x}) + e) \tag{5}$$

Where e represents the noise added during encryption to enhance security. The CKKS scheme utilizes various coefficient modulus bit sizes to effectively define its encryption parameters. In our implementation, we adopt the configuration: $[60, 40, 40, 60]$. This configuration plays a critical role in determining noise tolerance levels and significantly contributes to the overall security of the system. To recover the original plaintext from the ciphertexts, we employ a decryption function $\mathcal{D}$ that utilizes a private key sk:

$$\mathcal{D}(sk, \mathbf{c}) = \mathcal{E}(\mathbf{x}) + e \tag{6}$$

This decryption process yields a noisy version of the encoded plaintext, which can then be decoded to retrieve the original vector $\mathbf{x}$. In the proposed FL framework, clients first encrypt their model weights before transmitting them to a central server for aggregation. Each client follows a series of steps, starting with the flattening of their model weights into a single vector:

$$\mathbf{w}_i = [w_1^i, w_2^i, \ldots, w_d^i] \tag{7}$$

This transformation simplifies the subsequent encryption and aggregation processes. The flattened vector is then encrypted using the CKKS scheme, yielding:

$$\mathbf{c}_i = \text{CKKS}(\mathbf{w}_i) \tag{8}$$

After the server aggregates these encrypted weights into a single ciphertext, clients can decrypt the result to obtain their updated model weights. This approach enables clients to continue training their local models based on aggregated data while keeping their raw weights confidential. The server plays a critical role in this process by collecting encrypted weights from all participating clients and performing homomorphic addition to aggregate them:

$$\mathbf{c}_{\text{agg}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{c}_i \tag{9}$$

Here, $\mathbf{c}_{\text{agg}}$ represents the aggregated ciphertext that corresponds to the average of all received encrypted weights. This secure aggregation method allows for model updates without requiring decryption of individual client data. Mathematically, the Federated Learning process can be summarized as follows: Each client encrypts their model weights:

$$c_i^t = Enc(pk, w_i^t) \tag{10}$$

The server then aggregates these encrypted weights:

$$c_{\text{agg}}^t = \frac{1}{N} \sum_{i=1}^{N} c_i^t \tag{11}$$

Finally, each client decrypts the aggregated ciphertext to retrieve their updated model weights:

$$w_{\text{agg}}^t = \mathcal{D}(sk, c_{\text{agg}}^t) \tag{12}$$

3.9 Implementation of SFEDRL-IDS framework

We have implemented the solution using open-source software. The NIDS software architecture is depicted in Fig. 3, consisting of a network traffic analysis (NTA) developed using the Python Scapy library, capable of decoding packets from various protocols. To enhance performance, we extract and store network flow statistics in an in-memory database called Redis. This solution ensures faster access times as data resides in RAM, reducing the need for frequent access to the hard disk. For the critical task of persistent storage, including IDS configurations and information about detected anomalies, the chosen database is MongoDB, a robust NoSQL solution. This ensures data integrity even in situations like system failures or reboots. To further enhance accuracy and efficiency, we have developed our Anomaly Detection Model (ADM) using TensorFlow, a cutting-edge free and open-source software library for machine learning. Redis seamlessly integrates with the ADM, using its inputs to classify network flows effectively and trigger real-time notifications through Firebase Cloud Messaging. This ensures that all end-users receive timely alerts about potential security threats.

4 Experimentation and results

In this section, we will assess the performance of the SFEDRL-IDS framework by employing distinct neural network architectures for the online and target networks. This evaluation will assist in identifying the most effective model for deployment. Additionally, we will measure the resource consumption of the proposed framework using INTEL-IRRIS starter kits, which consist of low-cost equipment. This assessment aims to demonstrate the framework's efficiency without causing any degradation to the overall system.

4.1 Experimentation setup

In this study, the distributed algorithm 2 was executed using the RaspBerry Pi 4 model 4GB platform, which provided limited computational resources. A Quad-core ARM Cortex-A72 CPU, operating at 1.5 GHz, equips the board. The hyperparameters used in the experimentation are presented in Table 4.

This study represents a significant advancement of our foundational research on IoT applications in agriculture [20, 28, 29], rigorously enhancing established methodologies to set a new benchmark for IoT-driven agricultural solutions. Using the INTEL-IRRIS starter kit for evaluation (as shown in Fig. 4), our approach demonstrates not only practical applicability but also transformative potential, pushing the boundaries of innovation in the field [30], as illustrated in Fig. 4. We deploy an IoT-based smart irrigation system in an agricultural field, which includes a source node with soil moisture and temperature sensors and a LoRa communication module. The Raspberry Pi 4 serves as an IoT gateway for edge processing. The gateway has an embedded web application for monitoring and irrigation

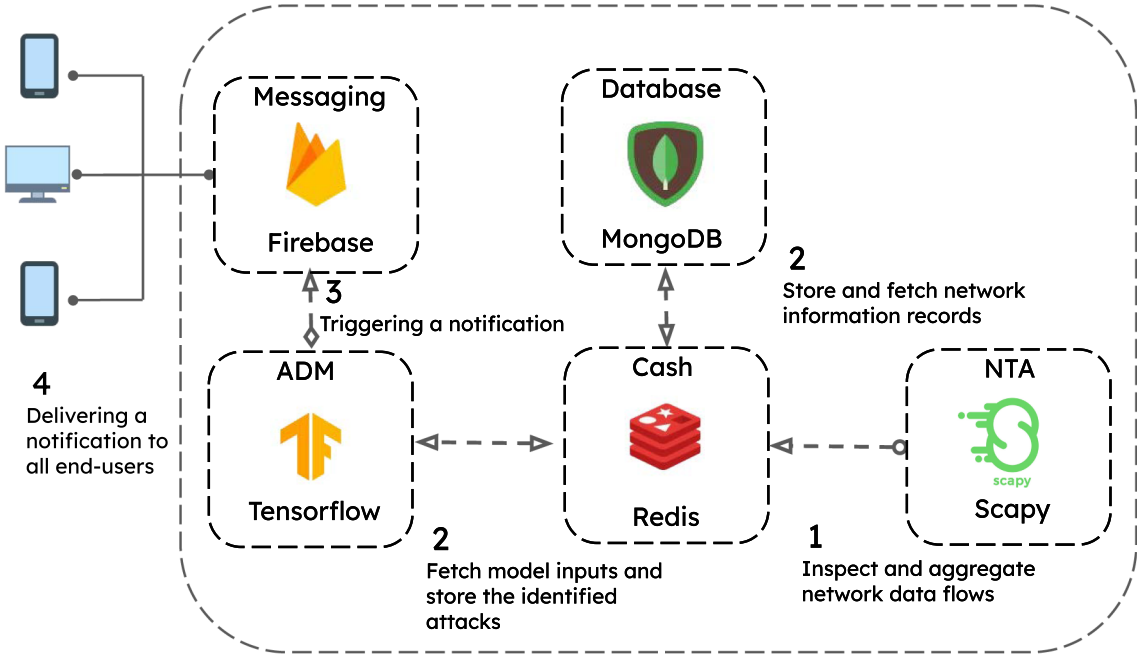


Fig. 3 Software architecture of SFEDRL-IDS

Table 4 Hyperparameters for experimentation

Hyperparameter	Value
Learning rate	0.0005
Epsilon (Epsilon Greedy)	0.5
Discount factor	0.05
Target network update frequency	3 episodes
Batch size	128
Steps per episode	1000
Replay buffer aize	50,000
Target network weights sharing frequency	6 episodes
FL rounds	20–30 rounds

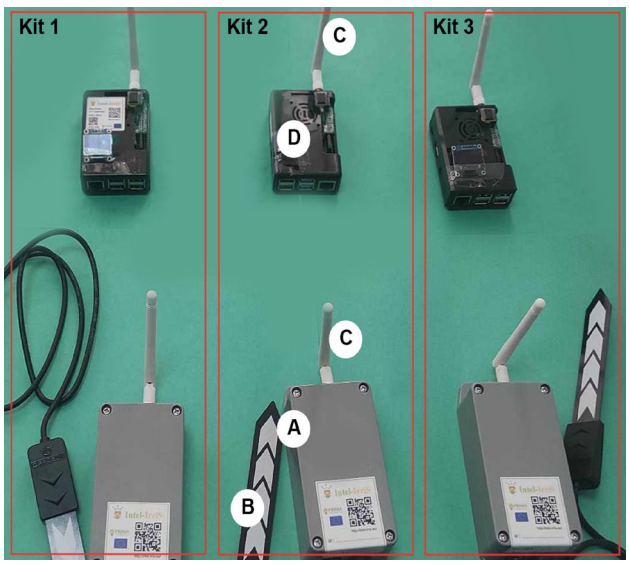


Fig. 4 Practical deployment of IDS in INTEL-IRRIS. Starter Kits: **A** microcontroller, **B** soil moisture sensor, **C** antenna, **D** IoT gateway

decision-making. The IoT gateway will deploy the proposed NIDS solution due to its connection to the internet network.

4.2 Performance metrics

The metrics for evaluating the IDS models include precision, recall, F1 score, and accuracy. They are calculated as follows:

$$\text{Accuracy} = \frac{T_p + T_n}{T_p + f_n + f_p + T_n} \tag{13}$$

$$\text{Precision} = \frac{T_p}{T_p + f_p} \tag{14}$$

$$\text{Recall} = \frac{T_p}{T_p + f_n} \tag{15}$$

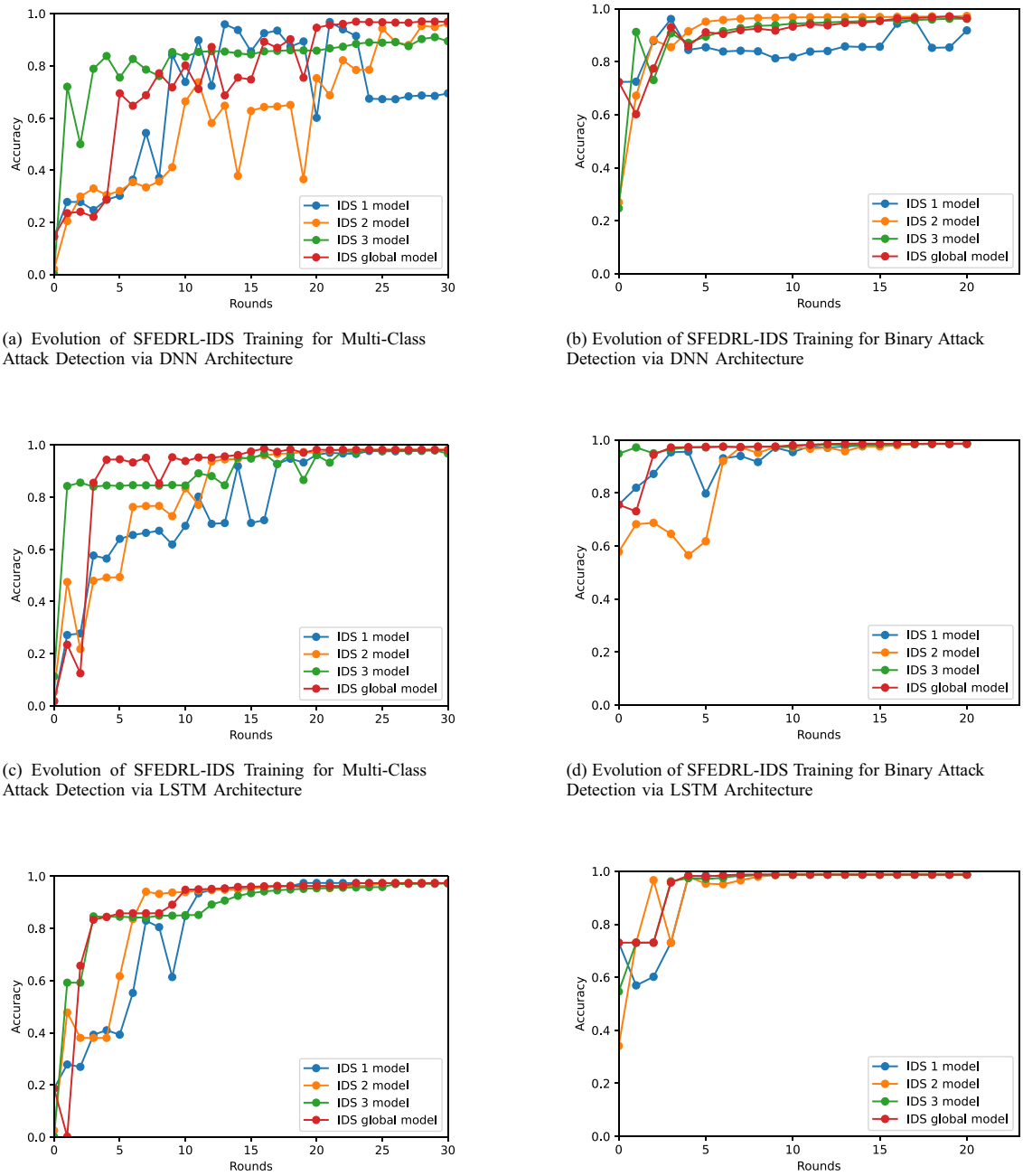
$$\text{F1} - \text{score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{16}$$

- True Positive (TP): represents the number of abnormal samples correctly classified as attacks (correct detection).
- False Positive (FP): represents the number of normal samples wrongly classified as attacks (incorrect detection).
- True Negative (TN): represents the number of normal samples wrongly classified (correct detection).
- False Negative (FN): represents the number of abnormal samples wrongly classified as attacks (incorrect detection).
- Accuracy: reports the proportion of properly categorized samples to all other samples in the testing set.
- Precision: reports the percentage of samples properly categorized for all TP and FP in the testing set.
- Recall: reports the ratio of TP samples to the total number of TP and FN samples is known as recall.
- The F1-score reports the harmonic mean between precision and recall.

4.3 Results and discussion

Initially, the accuracy of the IDS agent models is low due to the random initialization of the neural network weights. However, their performance progressively improves with each training round. Nonetheless, as training continues with Deep Neural Networks (DNNs), instabilities arise in the later rounds, as shown in Fig. 5a. The complexity of the temporal network data leads agents to both overfit and underfit their local environments, creating challenges for the effective comprehension of the DNN network.

As illustrated in Fig. 5b, the global model accuracy gradually increases, reaching an impressive 98.01% by round 30. This indicates the model's capacity to capture complex patterns after a prolonged training period. In contrast, the Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN)-based agents demonstrate a faster accuracy growth and achieve convergence earlier than the DNN, with accuracy rates of 98.49% and 98.67% at round 25, respectively. As shown in Fig. 5c and e, these models exhibit stability and cooperative convergence, showcasing their efficiency in handling multivariate time series data. The superior performance of the LSTM and CNN models underscores their potential to effectively capture temporal patterns and manage complex multivariate data. Leveraging these strengths could result



(a) Evolution of SFEDRL-IDS Training for Multi-Class Attack Detection via DNN Architecture

(b) Evolution of SFEDRL-IDS Training for Binary Attack Detection via DNN Architecture

(c) Evolution of SFEDRL-IDS Training for Multi-Class Attack Detection via LSTM Architecture

(d) Evolution of SFEDRL-IDS Training for Binary Attack Detection via LSTM Architecture

(e) Evolution of SFEDRL-IDS Training for Multi-Class Attack Detection via CNN Architecture

(f) Evolution of SFEDRL-IDS Training for Binary Attack Detection via CNN Architecture

Fig. 5 Training evolution for NIDS for multi-class and binary NIDS using different DDQN target model architectures

in higher accuracy rates and help mitigate the overfitting and underfitting issues encountered with DNNs. In conclusion, the results suggest that LSTM and CNN-based models are promising options for intrusion detection system (IDS) tasks when compared to DNNs. Furthermore, in terms of computation and inference time, CNN outperforms LSTM, making it more suitable for deployment in real IoT networks.

Table 5 comprehensively evaluates different deep reinforcement models in centralized and federated learning approaches for various types of attacks in an IDS. The table shows how well each attack type and IDS model (IDS1, IDS2, IDS3, and FLIDS) worked with DNN, LSTM, and CNN architectures, including precision, recall, and F1 Score. The testing of deep reinforcement models in the IDS shows that DNN-based models have low accuracy and recall for most types of attacks, which leads to lower

Table 5 SFEDRL-IDS performance for both centralized and federated learning approach

Attacks type	Precision				Recall				F1 Score			
	IDS1	IDS2	IDS3	FLIDS	IDS1	IDS2	IDS3	FLIDS	IDS1	IDS2	IDS3	FLIDS
Normal												
DNN	0.3972	0.962	0.9905	0.9713	0.9879	0.4317	0.871	0.9883	0.5666	0.596	0.9269	0.9797
LSTM	0.58517	0.4618	0.9552	0.9991	0.9588	0.9584	0.9862	0.955	0.7267	0.6232	0.9705	0.9766
CNN	0.5643	0.4702	0.9590	0.9986	0.96	0.9908	0.9733	0.9567	0.7108	0.6378	0.966	0.9772
Backdoor												
DNN	0.0	0.213	0.5136	0.9311	0.0	0.999	1.0	1.0	0.0	0.351	0.6786	0.9643
LSTM	0.0	0.9945	0.5778	0.99	0.0	,0.999	1.0	1.0	0.0	0.9968	0.732	0.9950
CNN	0.0	0.9488	0.5673	0.9177	0.0	1.0	1.0	0.9981	0.0	0.9737	0.7239	0.9562
DDoS HTTP Flooding												
DNN	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
LSTM	0.0255	0.0	0.0	0.9411	0.9272	0.0	0.0	0.8727	0.04968	0.0	0.0	0.9056
CNN	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.8545	0.0	0.0	0.0	0.9215
DDoS ICMP Flooding												
DNN	1.0	0.0	0.0	1.0	0.9878	0.0	0.0	0.9756	0.9938	0.0	0.0	0.9876
LSTM	0.9878	0.0	0.0	1.0	0.9878	0.0	0.0	0.9756	0.9878	0.0	0.0	0.9876
CNN	0.5744	0.0	0.0	1.0	0.9878	0.0	0.0	0.9878	0.7264	0.0	0.0	0.9938
DDoS TCP SYN Flooding												
DNN	0.4008	0.0	0.0	0.9892	0.9897	0.0	0.0	0.9387	0.5705	0.0	0.0	0.9633
LSTM	0.0956	0.0	0.0	1.0	1.0	0.0	0.0	0.9897	0.1745	0.0	0.0	0.9948
CNN	0.021	0.0	0.0	0.9898	1.0	0.0	0.0	1.0	0.041	0.0	0.0	0.9949
MITM												
DNN	0.0	0.7283	0.0	0.9672	0.0	0.9833	0.0	0.9833	0.0	0.8368	0.0	0.9752
LSTM	0.0	0.2438	0.0	0.9090	0.0	0.983	0.0	1.0	0.0	0.390	0.0	0.9523
CNN	0.0	0.4068	0.0	0.9672	0.0	0.9833	0.0	0.9833	0.0	0.5756	0.0	0.9752
Password												
DNN	0.0	0.0	0.9809	0.9996	0.0	0.0	0.9996	0.9952	0.0	0.0	0.9902	0.9974
LSTM	0.0	0.0	0.9790	0.9988	0.0	0.0	1.0	0.9988	0.0	0.0	0.9894	0.9988
CNN	0.0	0.0	0.9787	0.9988	0.0	0.0	1.0	1.0	0.0	0.0	0.9892	0.9994
Port Scanning												
DNN	0.0	0.0	0.8254	0.9877	0.0	0.0	0.9988	0.9976	0.0	0.0	0.9039	0.9926
LSTM	0.0	0.0	0.8854	0.9397	0.0	0.0	1.0	1.0	0.0	0.0	0.9392	0.9689
CNN	0.0	0.0	0.8816	0.9947	0.0	0.0	0.9988	0.9994	0.0	0.0	0.9365	0.997
Ransomware												
DNN	0.0	0.3313	0.0	1.0	0.0	1.0	0.0	0.9206	0.0	0.4978	0.0	0.9586
LSTM	0.3358	0.0	0.0	1.0	0.0	0.9988	0.0	0.9848	0.0	0.5026	0.0	0.9923
CNN	0.0	0.3406	0.0	0.9976	0.0	1.0	0.0	0.9988	0.0	0.5081	0.0	0.9982

F1 scores. In contrast, LSTM-based IDS models perform better than DNN, achieving higher precision, recall, and F1 scores, highlighting their effectiveness in capturing temporal patterns in the data. Similarly, CNN-based IDS models display competitive performance, particularly regarding precision and recall. Moreover, the results show that FLIDS generally achieves comparable or superior performance to individual centralized IDS models, especially for LSTM and CNN architectures. This shows the possible benefits of using federated learning to improve IDS performance, since the global model of FL can detect all kinds of attacks. In contrast, centralized learning IDS models may struggle to identify unknown attacks, as reflected by the null values in the corresponding cells of the table. Generally, centralized and federated learning approaches use deep reinforcement models for different

types of attacks in an IDS. The initial lower accuracy of the IDS agents models, resulting from random weights in the neural network, improves gradually with each training round. However, continuous training with DNNs leads to final round instabilities due to agents overfitting local environments. As shown in Fig. 9, the global model accuracy steadily increases, reaching an impressive 96.84% accuracy by round 20. On the other hand, LSTM and CNN-based agents get more accurate faster and converge earlier than DNN, reaching 98.49% accuracy at round 15 and 98.67% accuracy at round 10. These models demonstrate stability and cooperative convergence, highlighting their efficiency in handling multivariate time series data. The superior performance of LSTM and CNN models underscores their potential for effectively capturing temporal patterns and managing complex multivariate data. The results show that CNN-based models are a better choice for IDS tasks than DNNs and LSTMs, especially when you look at how long it takes to compute and draw conclusions for a real IoT network deployment. The table shows how well different deep reinforcement models work in both centralized and federated learning scenarios to defend against different types of attacks in an IDS. We check the accuracy, recall, and F1 Score for every type of attack and every IDS model (IDS1, IDS2, IDS3, and FLIDS) using DNN, LSTM, and CNN architectures. In normal network traffic, DNN-based IDS models demonstrate moderate precision values ranging from 0.4022 to 0.9246 (see Tables 6 and 7).

In contrast, their recall values are relatively high, ranging from 0.8925 to 0.9983, leading to F1 scores ranging from 0.5734 to 0.9404. On the other hand, LSTM-based IDS models perform better with precision values ranging from 0.4487 to 0.9349 and recall values ranging from 0.9206 to 1.0, resulting in F1 scores ranging from 0.6132 to 0.9732. The CNN-based IDS model, however, achieves lower precision and recall for normal traffic, leading to an F1 score of 0.6999. In terms of IoT network attacks, DNN-based IDS models have high precision values ranging from 0.9608 to 0.998, with recall values ranging from 0.4532 to 0.9849, resulting in F1 scores ranging from 0.6234 to 0.9785. LSTM-based IDS models show high precision values ranging from 0.9709 to 0.9781, but some have a recall value of 0, leading to F1 scores ranging from 0.7139 to 0.9904. The CNN-based IDS model achieves precision values ranging from 0.7307 to 0.9829 while consistently maintaining a recall value of 1.0, resulting in F1 scores ranging from 0.8444 to 0.9913. When you look at the performance of centralized and federated learning methods side by side, FLIDS usually does as well as or better than individual centralized IDS models, especially for LSTM and CNN architectures. This suggests the potential benefits of federated learning for enhancing IDS performance, especially in scenarios involving isolated environments.

The confusion matrix presented in Fig. 6 evaluates the performance of three Intrusion Detection Systems (IDS-1, IDS-2, and IDS-3) in classifying various network intrusions, each designed to address specific attack scenarios with differing accuracy and misclassification rates. IDS-1 demonstrates high accuracy in detecting "Backdoor" and "Password attacks," with 548 and 1,129 correct predictions, respectively. It also effectively identifies "Port Scanning attacks," achieving 845 correct classifications, while misclassifying only a small number of normal instances and attacks. IDS-2 specializes in DDoS attack detection and performs well across multiple DDoS scenarios (see Figs. 7, 8, 9, 10)

It accurately classifies "Normal" traffic with 97 instances and various DDoS flood attacks, including "DDoS TCP SYN Flood" (32 correct) and "DDoS UDP Flood" (26 correct). Overall, IDS-2 effectively distinguishes between normal traffic and DDoS attacks. By

Table 6 SFEDRL-IDS performance for both centralized and federated learning approach

Attacks type	Precision				Recall				F1 Score			
	IDS1	IDS2	IDS3	FLIDS	IDS1	IDS2	IDS3	FLIDS	IDS1	IDS2	IDS3	FLIDS
Normal												
DNN	0.4022	0.455	0.9246	0.9575	0.9983	0.9611	0.8925	0.9239	0.5734	0.6176	0.9082	0.9404
LSTM	0.54676	0.4487	0.9349	1.0	0.957	0.9681	0.9206	0.9479	0.6959	0.6132	0.9277	0.9732
CNN	0.5524	0.0	0.0	1.0	0.9549	0.08	0.0	0.9528	0.6999	0.0	0.0	0.9758
Attack												
DNN	0.998	0.9757	0.9608	0.9723	0.4532	0.5758	0.9731	0.9849	0.6234	0.7242	0.9669	0.9785
LSTM	0.9781	0.9795	0	0.97091	0.7076	0.5617	0.976	1.0	0.8211	0.7139	0.9736	0.9904
CNN	0.9773	0.7307	0.7307	0.9829	0.7149	1.0	1.0	1.0	0.8257	0.8444	0.8444	0.9913

Table 7 Class-specific accuracy comparison across studies by Tareq et al. [31], Ferrag et al. [8], Elias et al. [32], and our research

Attack type	References [31]	References [8]	References [32]	SFEDRL-IDS
Backdoor	98%	86%	79%	99.5%
DDOS HTTP	95%	92%	100%	92.15%
DDOS ICMP	100%	99%	94%	99.38%
DDOS TCP	100%	100%	100%	99.49%
MITM	100%	100%	96%	97.52%
Normal	100%	100%	74%	97.72%
Password	19%	38%	100%	99.94%
Port Scanning	49%	74%	73%	99.70%
Ransomware	88%	85%	99%	99.82%

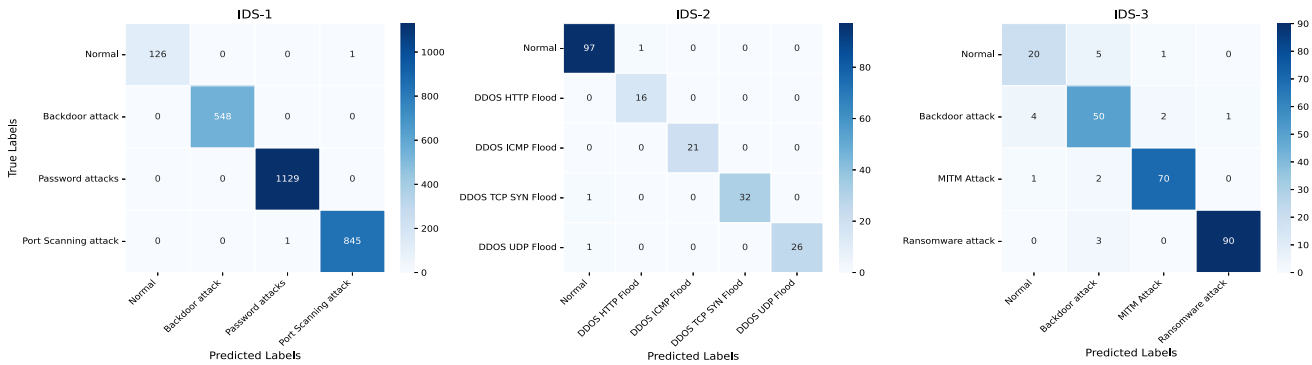


Fig. 6 Confusion matrix visualization of NIDS model performance during the deployment phase

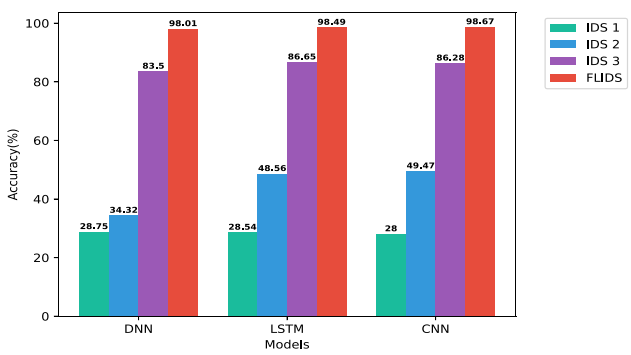


Fig. 7 Performance of multiclass IDS models with centralized and federated learning DDQN under varied model architectures

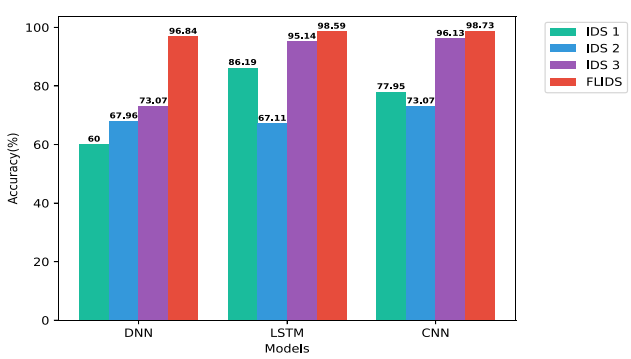


Fig. 8 Performance of binary IDS Models with centralized and federated learning DDQN under varied model architectures

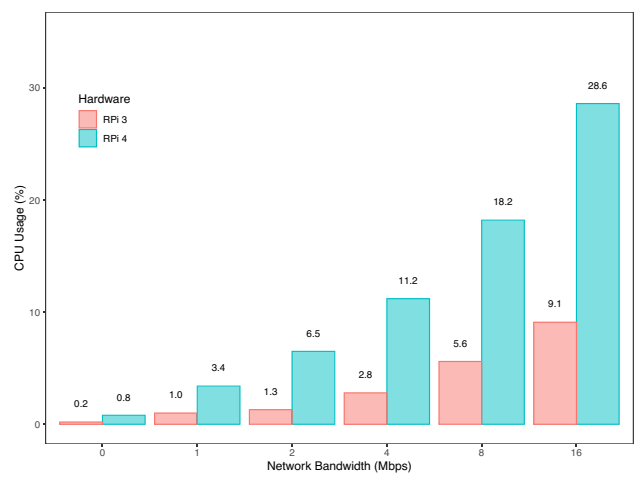


Fig. 9 Analysis of CPU utilization in IDS on raspberry Pi 3 and 4 across diverse network bandwidths

incorporating secure exchanges and aggregation of local model weights, SFEDRL-IDS consistently achieves high accuracy across most attack types, reaching 99.5% for Backdoor attacks and over 99% for detecting Password and Port Scanning attacks.

Additionally, the model demonstrates strong performance in complex categories, such as Ransomware and DDOS TCP. These results indicate that SFEDRL-IDS significantly enhances detection capabilities across diverse

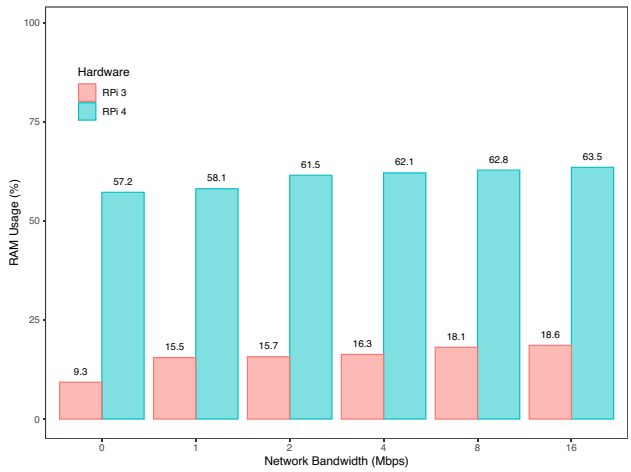


Fig. 10 Analysis of RAM utilization in IDS on raspberry Pi 3 and 4 across diverse network bandwidths

attack scenarios. We deploy the IDS at the IoT gateways with the WLAN network interface to analyze incoming and outgoing traffic. We test the proposed solution's performance in various scenarios by varying network bandwidth using the iperf3 tool, which generates network traffic with different protocols (TCP or UDP). We conduct the evaluation on the Raspberry Pi 3 Model B, which boasts a 64-bit quad-core ARM Cortex-A53 CPU operating at 1.2 GHz, and the Raspberry Pi 4 Model B (4GB RAM variant), equipped with a more powerful 64-bit quad-core ARM Cortex-A72 CPU operating at 1.5 GHz. The results indicate that both RPi3 and RPi4 boards exhibit low CPU usage across all scenarios, but RPi4 demonstrates lower CPU usage (9.1%) compared to RPi3 (28.6%) at 16 Mbps. We can attribute this to the fact that RPi4 has more available resources. However, the memory usage is initially higher for RPi3 (57.2%) due to its limited total memory and the consumption of memory by OS processes. As the network bandwidth increases, RAM usage also increases, reaching 63.5% for RPi3 and 16.6% for RPi4, respectively. Evaluating the IDS deployed on the Raspberry Pi 3 Model B and Raspberry Pi 4 Model B (4GB RAM variant) as IoT gateways reveals promising results. Both Raspberry Pi models show low CPU usage, indicating an efficient IDS implementation. The Raspberry Pi 4 demonstrates superior CPU performance, achieving significantly lower CPU usage at high network bandwidth than the Raspberry Pi 3. However, both models experience increasing RAM usage as network bandwidth rises, with the Raspberry Pi 3 starting at higher memory consumption due to its limited capacity. Overall, the study confirms that both RPi models can effectively function as IoT gateway NIDS devices, with the Raspberry Pi 4 showing better performance in CPU-intensive scenarios. Memory management remains crucial, especially for the Raspberry Pi 3, with its limited RAM.

4.4 Computational cost of CKKS scheme

The implementation of the CKKS scheme incurs several computational costs linked to encryption, decryption, and homomorphic operations, including addition and multiplication. The encryption cost for CKKS is $O(N \cdot \log(N))$ per vector component, where N is the polynomial modulus degree. This complexity arises from the polynomial operations needed to encrypt a vector of size n. Key factors affecting this cost include the polynomial modulus degree (set to 8192 and 4096 in our experiments) and the bit size of the coefficient modulus. Additionally, the global scale can vary, such as 2^{21}. The encryption cost for each model weight depends on the weight vector size W and other parameters, approximated as $O(W \cdot N \cdot \log(N))$.

Similarly, the decryption cost demonstrates a comparable complexity, scaling as $O(W \cdot N \cdot \log(N))$. This similarity indicates that both encryption and decryption processes require similar computational resources, especially when handling encrypted weights in federated learning. Figure 11 displays the encryption and decryption times, measured in milliseconds, for the three classifiers utilizing the CKKS homomorphic encryption scheme. Among these, CNN exhibits the longest encryption time at 1286.51 ms, followed by DNN at 1061.67 ms, with LSTM being the fastest at 466.04 ms. These variations in encryption times can be attributed to the differing complexities of each model, specifically their parameter counts: DNN has 287,275 parameters, LSTM has 124,043 parameters, and CNN has 341,419 parameters. Encryption takes longer than decryption due to the CKKS scheme's requirement for complex mathematical operations to secure data. This process includes polynomial approximations and

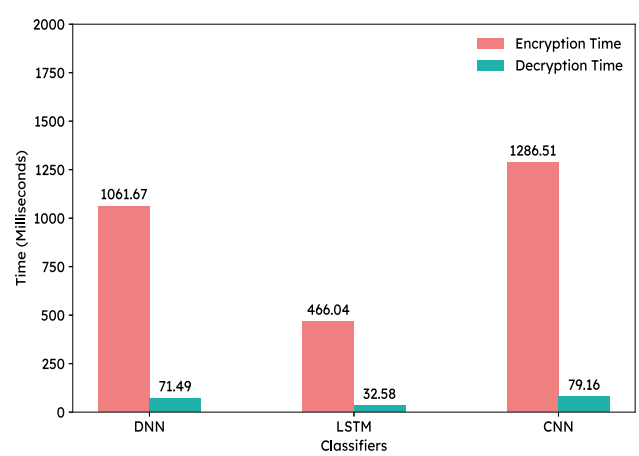


Fig. 11 Evaluation of CKKS scheme encryption and decryption times for different classifier models

modulus switching, both of which are computationally intensive to ensure that encrypted data can be processed without decryption. In contrast, decryption involves fewer operations, typically consisting of simpler transformations to recover the original data from its encrypted state. As a result, decryption is significantly faster across all models. LSTM records the shortest decryption time at 32.58 ms, while CNN takes the longest at 79.16 ms. The quicker decryption for LSTM may be linked to its lower parameter count, which reduces the volume of data needing decryption compared to CNN and DNN. In summary, encryption represents the primary computational cost, particularly for complex models with a large number of parameters in constrained IoT devices .

4.5 Analyzing communication costs in CKKS for encrypted weight transmission

The CKKS encryption process significantly increases the size of model weights, resulting in larger data transfers between clients and the server compared to the original plaintext size. The size of the encrypted data is determined by several factors, including the polynomial modulus degree N, which influences the number of coefficients in the polynomial, and the coefficient modulus bit size. The size of each coefficient is calculated by dividing the total bit size of this list by 8 to convert it to bytes. Additionally, the weight vector size W indicates the number of parameters being encrypted .

For CKKS, the plaintext weight size per vector can be estimated as $W \times 4$ bytes, assuming a 32-bit floating-point representation. However, the size of the encrypted weights is considerably larger, estimated at approximately $W \times K \times \text{bytes_per_coefficient}$, where K represents the ciphertext expansion factor. Consequently, the formula for the total size of encrypted weights per client can be expressed as:

$$\text{Size}_{\text{encrypted}} \approx W \times K \times (\text{bytes per coefficient}). \quad (17)$$

Ultimately, for C clients, the total communication cost for a round of data transfer can be summarized as:

$$\text{Total Communication Cost} = C \times \text{Size}_{\text{encrypted}}. \quad (18)$$

Figure 12 illustrates the weight sizes in kilobytes (KB) for three classifiers before and after applying CKKS encryption. Across all classifiers, the encrypted weights are significantly larger, indicating a considerable encryption overhead.

For the CNN, the weight size increases from 1,333.67 KB in plaintext to 15,685.26 KB when encrypted, representing the most substantial growth among the classifiers. This suggests that models with a large number of parameters, such as CNNs, experience greater overhead. In the case of the LSTM classifier, the weight size expands from 484.54 KB to 5,788.44 KB after encryption, resulting in a twelvefold increase. Although it has fewer parameters than the CNN, the LSTM still incurs significant overhead. The DNN shows a similar trend, with its weight size rising from 1,122.17 KB to 13,255.64 KB after encryption, although at a slightly lower level than that of the CNN. This increase in communication overhead underscores the substantial growth in data size due to CKKS encryption. However, it's important to recognize that federated learning rounds are not conducted continuously but rather follow a periodic schedule. This approach reduces the frequency of large data transfers, thereby alleviating some pressure on network and storage resources. Consequently, CKKS remains an effective solution for safeguarding data privacy against untrusted aggregators.

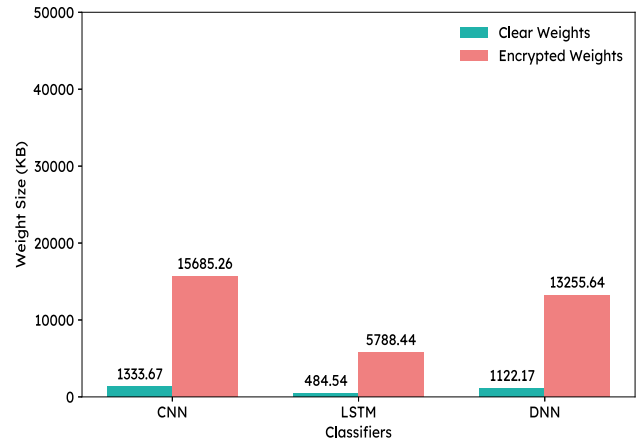


Fig. 12 CKKS scheme payload overhead analysis for various classifier architectures

5 Conclusions and future work

This research paper presents the development of a cutting-edge distributed IoT intrusion detection system that prioritizes the preservation of data privacy. By leveraging deep federated reinforcement learning (DFRL) within isolated environments, our approach achieves remarkable enhancements in training efficiency and accuracy. The CNN-based DDQN networks attained outstanding accuracy rates of 98.67% and 98.73%, significantly outperforming conventional DNN and LSTM architectures while maintaining stringent data integrity. Rigorous testing of the proposed solution demonstrates its effectiveness and suitability, ensuring no degradation in the performance of the IoT Intel-Irris smart irrigation system. The findings illuminate the substantial efficacy of the distributed approach in bolstering intrusion detection capabilities in IoT

environments while safeguarding sensitive data. In summary, this research contributes critical insights into the realms of IoT security and privacy, presenting a robust and efficient solution for intrusion detection in distributed systems. The impressive accuracy rates not only suggest significant real-world applicability but also establish a promising foundation for future innovations in this vital field. The proposed solution is set to undergo significant enhancements and expansions in alignment with the advent of Industry 5.0, which aspires to establish a hyperconnected, automated, and data-driven industrial ecosystem [33]. While traditional deep learning-based intrusion detection systems (IDS) demonstrate commendable accuracy, they are often hindered by the "black box" problem, which erodes user trust. In response, researchers are pioneering explainable and intelligent mechanisms for data-efficient intrusion detection specifically designed for the Industrial Internet of Things (IIoT) [34]. Furthermore, upcoming iterations of this solution will integrate cutting-edge IoT blockchain technology along with smart contract-enabled access control and authentication mechanisms, cultivating trust among participating entities within a Zero Touch Network (ZTN) [35]. This integration is poised to substantially enhance the security and integrity of data within the IoT ecosystem. Moreover, we will proactively tackle model poisoning attacks within the federated learning framework for Network Intrusion Detection Systems (NIDS) by detecting non-independent and identically distributed encrypted gradients. We will conduct a comparative analysis of the CKKS scheme against other leading methods from the literature, including Additive Homomorphic Encryption and Two-Trapdoor Homomorphic Encryption [36, 37]. These future endeavors are expected to significantly elevate the effectiveness and practicality of the proposed solution, directly addressing critical aspects of IoT security and contributing to the overall advancement of the field. The ongoing research and implementation of these enhancements promise immense potential for cultivating more secure and resilient IoT environments.

Acknowledgements We owe an immense debt of gratitude to Pr. Congduc Pham, from the Department of Computer Science at the University of Pau (France), for his unparalleled support and steadfast guidance throughout this project. His extraordinary expertise, meticulous attention to detail, and tireless assistance in assembling the kits and providing technical insights were absolutely crucial to the success of our research. His generosity with his time, profound knowledge, and inspiring encouragement not only elevated the quality of our work but also propelled us forward at every stage. This project would not have been possible without his invaluable contributions.

Funding This work was supported by the INTEL-IRRIS-PRIMA S2 2020 Project (ID 1560) (https://intel-irris.eu/).

Data availability Enquiries about data availability should be directed to the authors.

Declarations

Conflict of interest The authors declare no Conflict of interest or Conflict of interest related to this research. The research was conducted without any financial or personal relationships that could be perceived as influencing the work presented.

References

1. Mekala, S.H., Baig, Z., Anwar, A., Zeadally, S.: Cybersecurity for industrial IoT (IIoT): threats, countermeasures, challenges and future directions. Comput. Commun. **208**, 294–320 (2023). https://doi.org/10.1016/j.comcom.2023.06.020
2. Choo, K.-K.R., Gai, K., Chiaraviglio, L., Yang, Q.: A multidisciplinary approach to internet of things (IoT) cybersecurity and risk management. Comput. Secur. **102**, 102136 (2021). https://doi.org/10.1016/j.cose.2020.102136
3. Yazdinejadna, A., Parizi, R.M., Dehghantanha, A., Khan, M.S.: A kangaroo-based intrusion detection system on software-defined networks. Comput. Netw. **184**, 107688 (2021)
4. Tharewal, S., Ashfaque, M.W., Banu, S.S., Uma, P., Hassen, S.M., Shabaz, M.: Intrusion detection system for industrial internet of things based on deep reinforcement learning. Wirel. Commun. Mob. Comput. **2022**, 1–8 (2022)
5. Suwannalai, E., Polprasert, C.: Network intrusion detection systems using adversarial reinforcement learning with deep q-network. In: 2020 18th International Conference on ICT and Knowledge Engineering (ICT). IEEE, (2020)
6. Sadiki, A., Bentahar, J., Dssouli, R., En-Nouaary, A., Otrok, H.: Deep reinforcement learning for the computation offloading in MIMO-based edge computing. Ad Hoc Netw. **141**, 103080 (2023)
7. Giménez, N.L., Solé, J.M., Freitag, F.: Embedded federated learning over a LoRa mesh network. Pervasive Mob. Comput. **93**, 101819 (2023)
8. Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L., Janicke, H.: Edge-IIoTset: a new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning. (2022)
9. Elrawy, M.F., Awad, A.I., Hamed, H.F.A.: Intrusion detection systems for IoT-based smart environments: a survey. J. Cloud Comput. **7**(1), 1–20 (2018)
10. Zarpelão, B.B., Miani, R.S., Kawakani, C.T., de Alvarenga, S.C.: A survey of intrusion detection in internet of things. J. Netw. Comput. Appl. **84**, 25–37 (2017)
11. Zhang, Y., Liu, Q.: On IoT intrusion detection based on data augmentation for enhancing learning on unbalanced samples. Futur. Gener. Comput. Syst. **133**, 213–227 (2022)
12. Chohra, A., Shirani, P., Karbab, E.B., Debbabi, M.: Chameleon: optimized feature selection using particle swarm optimization and ensemble methods for network anomaly detection. Comput. Secur. **117**, 102684 (2022)
13. Lee, J.-S., Chen, Y.-C., Chew, C.-J., Chen, C.-L., Huynh, T.-N., Kuo, C.-W.: CoNN-IDS: Intrusion detection system based on collaborative neural networks and agile training. Comput. Secur. **122**, 102908 (2022)
14. Javeed, D., Gao, T., Saeed, M.S., Kumar, P.: An intrusion detection system for edge-envisioned smart agriculture in extreme environment. IEEE Int. Things J. **11**(16), 26866–26876 (2024)

15. Friha, O., Ferrag, M.A., Shu, L., Maglaras, L., Choo, K.-K.R., Nafaa, M.: FELIDS: Federated learning-based intrusion detection system for agricultural internet of things. J. Parallel Distrib. Comput. **165**, 17–31 (2022)
16. Javeed, D., Saeed, M.S., Adil, M., Kumar, P., Jolfaei, A.: A federated learning-based zero trust intrusion detection system for internet of things. Ad Hoc Netw. **162**, 103540 (2024)
17. Benameur, R., Dahane, A., Souihi, S., Mellouk, A.: A novel federated learning based intrusion detection system for iot networks. In: ICC 2024 - IEEE International Conference on Communications. IEEE, pp. 2402–2407 (2024)
18. Yazdinejad, A., Dehghantanha, A., Parizi, R.M., Hammoudeh, M., Karimipour, H., Srivastava, G.: Block hunter: federated learning for cyber threat hunting in blockchain-based iiot networks. IEEE Trans. Industr. Inf. **18**(11), 8356–8366 (2022)
19. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A.: Application of deep reinforcement learning to intrusion detection for supervised problems. Expert Syst. Appl. **141**, 112963 (2020)
20. Benameur, R., Dahane, A., Kechar, B., Benyamina, A.E.H.: An innovative smart and sustainable low-cost irrigation system for anomaly detection using deep learning. Sensors **24**(4), 1162 (2024)
21. Zeng, Z., Cui, L., Qian, M., Zhang, Z., Wei, K.: A survey on sliding window sketch for network measurement. Comput. Netw. **226**, 109696 (2023)
22. Théate, T., Ernst, D.: Risk-sensitive policy with distributional reinforcement learning. Algorithms **16**(7), 325 (2023)
23. Arulkumaran, K., Deisenroth, M.P., Brundage, M., Bharath, A.A.: Deep reinforcement learning: a brief survey. IEEE Signal Process. Mag. **34**(6), 26–38 (2017)
24. Cai, Q., Cui, C., Xiong, Y., Wang, W., Xie, Z., Zhang, M.: A survey on deep reinforcement learning for data processing and analytics. IEEE Transactions on Knowledge and Data Engineering, 1–1 (2022)
25. Zhang, J., Liu, Z.: Perfreezeclip: personalized federated learning based on adaptive clipping. Electronics **13**(14), 2739 (2024)
26. Natarajan, D., Dai, W.: Seal-embedded: a homomorphic encryption library for the internet of things. IACR Transactions on Cryptographic Hardware and Embedded Systems, 756–779 (2021)
27. Behera, S., Prathuri, J.R.: Fpga-based acceleration of k-nearest neighbor algorithm on fully homomorphic encrypted data. Cryptography **8**(1), 8 (2024)
28. Dahane, A., Benameur, R., Kechar, B., Benyamina, A.: An iot based smart farming system using machine learning. In: 2020 International Symposium on Networks, Computers and Communications (ISNCC). Montreal, QC, Canada pp. 1–6 (2020)
29. Dahane, A., Benameur, R., Kechar, B.: An iot low-cost smart farming for enhancing irrigation efficiency of smallholder farmers. Wireless Pers. Commun. **127**(12), 3173–3210 (2022)
30. Pham, C., Rahim, A., Hartmann, C., Dupont, C., Forster, J., Markwordt, F., Printanier, J.-F., Kechar, B., Benkhelifa, M., Baraka, K., Benabdelouahab, T., Bartzanas, T., Fountas, S.: Deploying low-cost and full edge-IoT/AI system for optimizing irrigation in smallholder farmers communities. In: Ambient Intelligence and Smart Environments. IOS Press, (2022)
31. Tareq, I., Elbagoury, B.M., El-Regaily, S., El-Horbaty, E.-S.M.: Analysis of ton-iot, unw-nb15, and edge-iiot datasets using dl in cybersecurity for iot. Appl. Sci. **12**(19), 9572 (2022)
32. Elias, E.M.d., Carriel, V.S., De Oliveira, G.W., Dos Santos, A.L., Nogueira, M., Junior, R.H., Batista, D.M.: A hybrid cnn-lstm model for iiot edge privacy-aware intrusion detection. In: 2022 IEEE Latin-American Conference on Communications (LATINCOM). IEEE (Nov. 2022)
33. Javeed, D., Gao, T., Kumar, P., Jolfaei, A.: An explainable and resilient intrusion detection system for industry 5.0. IEEE Trans. Consum. Electron. **70**(1), 1342–1350 (2024)
34. Attique, D., Hao, W., Ping, W., Javeed, D., Kumar, P.: Explainable and data-efficient deep learning for enhanced attack detection in iiot ecosystem. IEEE Int Things J 1–1 (2024)
35. Kumar, R., Aljuhani, A., Javeed, D., Kumar, P., Islam, S., Islam, A.N.: Digital twins-enabled zero touch network: a smart contract and explainable ai integrated cybersecurity framework. Futur. Gener. Comput. Syst. **156**, 191–205 (2024)
36. Yazdinejad, A., Dehghantanha, A., Karimipour, H., Srivastava, G., Parizi, R.M.: A robust privacy-preserving federated learning model against model poisoning attacks. IEEE Trans. Inf. Forensics Secur. **19**, 6693–6708 (2024)
37. Yazdinejad, A., Dehghantanha, A., Srivastava, G., Karimipour, H., Parizi, R.M.: Hybrid privacy preserving federated learning against irregular users in next-generation internet of things. J. Syst. Architect. **148**, 103088 (2024)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Dr. Rabaie Benameur holds a Ph.D. in Cyber Security Systems and Networks, with extensive research experience in intelligent systems. His work primarily focuses on cutting-edge topics such as machine learning, deep learning, and federated learning. Over the years, Dr. Benameur has contributed to a significant number of peer-reviewed journal articles and high-profile conference proceedings, showcasing his expertise in both the theoretical and practical aspects of secure and intelligent systems. His contributions are recognized in domains that blend cybersecurity with advanced AI techniques, positioning him as a key researcher in the field.

Amine Dahane holds a Master's degree in Computer Systems and Networks from the University of Bechar (2011) and a Ph.D. in Electronics from the University of Science and Technology of Oran (USTO) (2017). He is also an associate member of the LiSSi/Tinc-NET Laboratory at the University of Paris-Est Créteil, France. A distinguished academic and researcher, Dr. Dahane's expertise spans a broad array of fields, including wireless sensor networks, cybersecurity, smart farming, machine learning, deep learning, federated learning, precision agriculture, and water quality monitoring. Currently, he is a lecturer and researcher at the Institute of Sciences and Applied Techniques, University of Oran 1, where his work focuses on the integration of intelligent systems to promote sustainability, particularly in agriculture and environmental monitoring.